

SephiroFlows User Guide

Table of Contents

1. [Getting Started](#)
2. [Creating Workflows](#)
3. [Node Reference](#)
4. [Expressions](#)
5. [Executing Workflows](#)
6. [Monitoring Executions](#)
7. [Best Practices](#)
8. [Troubleshooting](#)

Getting Started

First Login

1. Navigate to `http://localhost:5173`
2. Use demo credentials:
 - Email: `demo@sephiroflows.app`
 - Password: `demo123`
3. You'll be redirected to the Dashboard

Understanding the Interface

The application has four main sections:

- **Dashboard:** Overview of workflows and executions
- **Workflows:** Create and manage workflows
- **Executions:** View workflow execution history
- **User Menu:** Access settings and logout

Creating Workflows

Step 1: Create a New Workflow

1. Click **Workflows** in the sidebar
2. Click **New Workflow** button
3. You'll see the visual workflow editor

Step 2: Add Nodes

1. The **Node Library** appears on the left
2. Browse available nodes by category:
 - **Trigger:** Workflow entry points
 - **Action:** Perform operations
 - **Transform:** Manipulate data
 - **Logic:** Control flow
3. Click a node to add it to the canvas

4. Drag nodes to reposition them

Step 3: Connect Nodes

1. Hover over a node's output port (right side)
2. Click and drag to another node's input port (left side)
3. Release to create a connection
4. Connections represent data flow

Step 4: Configure Nodes

1. Click on a node to select it
2. The **Configuration Panel** appears on the right
3. Fill in required parameters (marked with *)
4. Use expressions for dynamic values
5. Click **Save Configuration**

Step 5: Save Workflow

1. Enter a workflow name at the top
2. Add a description (optional)
3. Click **Save** button
4. Your workflow is now saved as a draft

Node Reference

1. Manual Trigger

Category: Trigger

Purpose: Start workflow manually

Parameters: None

Output:

```
{
  "data": {}, // Trigger data passed during execution
  "timestamp": "2025-11-28T10:30:00Z"
}
```

Usage: Always start your workflow with a trigger node.

2. HTTP Request

Category: Action

Purpose: Make HTTP API calls

Parameters:

- `method` (required): GET, POST, PUT, DELETE, PATCH
- `url` (required): API endpoint URL
- `headers` : HTTP headers (JSON object)
- `body` : Request body (for POST/PUT/PATCH)
- `timeout` : Request timeout in ms (default: 30000)

Output:

```
{
  "status": 200,
  "statusText": "OK",
  "headers": {},
  "data": {} // Response body
}
```

Example:

```
{
  "method": "POST",
  "url": "https://api.example.com/users",
  "headers": {
    "Authorization": "Bearer {{$variables.apiKey}}",
    "Content-Type": "application/json"
  },
  "body": {
    "name": "{{$trigger.data.name}}",
    "email": "{{$trigger.data.email}}"
  }
}
```

3. Data Transform

Category: Transform**Purpose:** Transform and map data**Parameters:**

- `mapping` (required): Object mapping for transformation

Output: Transformed data based on mapping**Example:**

```
{
  "mapping": {
    "fullName": "{{$node.http1.data.firstName}} {{$node.http1.data.lastName}}",
    "email": "{{$node.http1.data.email}}",
    "isActive": true
  }
}
```

Output:

```
{
  "fullName": "John Doe",
  "email": "john@example.com",
  "isActive": true
}
```

4. Conditional Logic

Category: Logic

Purpose: Branch workflow based on conditions

Parameters:

- `condition` (required): Value to evaluate
- `operator`: equals, not_equals, greater_than, less_than, contains
- `value`: Value to compare against

Output:

```
{
  "result": true, // boolean
  "condition": "...",
  "operator": "equals",
  "value": "..."
}
```

Example:

```
{
  "condition": "{{ $node.http1.data.status }}",
  "operator": "equals",
  "value": "200"
}
```

5. Set Variable

Category: Logic

Purpose: Store values in workflow variables

Parameters:

- `name` (required): Variable name
- `value` (required): Variable value

Output:

```
{
  "name": "apiKey",
  "value": "...",
  "set": true
}
```

Usage: Store values to reuse in later nodes using `{{ $variables.name }}`

6. Send Email

Category: Action

Purpose: Send emails via SMTP

Parameters:

- `to` (required): Recipient email
- `subject` (required): Email subject
- `body` (required): Email body
- `from` : Sender email (defaults to SMTP_USER)
- `isHtml` : Whether body is HTML (default: false)

Output:

```
{
  "messageId": "<unique-id>",
  "accepted": ["recipient@example.com"],
  "rejected": [],
  "response": "250 OK"
}
```

Note: Requires SMTP configuration in backend/.env

Expressions

Syntax

Expressions use double curly braces: `{{expression}}`

Accessing Node Data

Reference output from previous nodes:

```
{{${node.nodeId}.data.field}}
```

Examples:

- `{{${node.http1.data.status}}}` - HTTP status code
- `{{${node.http1.data.body.user.name}}}` - Nested field
- `{{${node.transform1.email}}}` - Transformed data

Accessing Trigger Data

Reference data passed when starting the workflow:

```
{{${trigger.data.field}}
```

Examples:

- `{{${trigger.data.email}}}` - Email from trigger
- `{{${trigger.data.user.id}}}` - Nested trigger data

Using Variables

Reference workflow variables set by Set Variable nodes:

```
{{${variables.variableName}}}
```

Examples:

- `{{variables.apiKey}}` - API key variable
- `{{variables.userId}}` - User ID variable

Built-in Functions

Date/Time

- `{{now()}}` - Current ISO timestamp

String Manipulation

- `{{uppercase(value)}}` - Convert to uppercase
- `{{lowercase(value)}}` - Convert to lowercase

Utility

- `{{length(array)}}` - Get array/string/object length
- `{{json(string)}}` - Parse JSON string

Combining Expressions

You can use multiple expressions in one value:

```
"Hello {{node.http1.data.firstName}}, your status is {{node.http1.data.status}}"
```

Executing Workflows

Manual Execution

1. Open workflow in editor
2. Click **Execute** button
3. Optionally provide trigger data
4. Workflow starts immediately
5. You'll be redirected to execution details

Trigger Data

When executing manually, you can provide JSON data:

```
{
  "email": "user@example.com",
  "name": "John Doe",
  "plan": "premium"
}
```

This data is accessible via `{{trigger.data.field}}` in any node.

Monitoring Executions

Viewing All Executions

1. Click **Executions** in sidebar
2. See list of all workflow executions
3. Status indicators:
 -  Success - Completed successfully

-  Failed - Error occurred
-  Running - Currently executing
-  Queued - Waiting to start

Execution Details

1. Click on any execution
2. View execution overview:
 - Status and duration
 - Trigger information
 - Error messages (if failed)
3. See node-by-node logs:
 - Each node's status
 - Input/output data
 - Execution time
 - Error details

Understanding Logs

Each node run shows:

- **Status:** Success, Failed, Running, Pending
- **Duration:** How long the node took
- **Input Data:** Parameters passed to node
- **Output Data:** Result from node execution
- **Error Message:** If node failed

Best Practices

Workflow Design

1. **Start Simple:** Begin with basic workflows
2. **Test Incrementally:** Add and test one node at a time
3. **Use Clear Names:** Name nodes descriptively
4. **Add Comments:** Use node labels to document logic
5. **Handle Errors:** Plan for failure scenarios

Node Configuration

1. **Validate URLs:** Ensure API endpoints are correct
2. **Use Variables:** Store reusable values
3. **Test Expressions:** Verify expression syntax
4. **Check Types:** Match data types (string, number, boolean)
5. **Set Timeouts:** Configure appropriate timeouts

Performance

1. **Minimize HTTP Calls:** Batch requests when possible
2. **Use Transforms:** Process data efficiently
3. **Limit Loops:** Avoid infinite loops
4. **Monitor Execution Time:** Keep workflows fast
5. **Clean Up:** Delete unused workflows

Security

1. **Protect Credentials:** Use environment variables
2. **Validate Input:** Sanitize trigger data
3. **Use HTTPS:** Always use secure connections
4. **Review Logs:** Check for sensitive data exposure
5. **Limit Access:** Control who can edit workflows

Troubleshooting

Common Issues

Workflow Won't Execute

Problem: "Failed to execute workflow"

Solutions:

- Check all nodes have required parameters
- Verify nodes are connected in order
- Ensure workflow has a trigger node
- Check for circular dependencies

Expression Not Working

Problem: Expression returns undefined

Solutions:

- Verify node ID matches
- Check data path is correct
- Ensure previous node executed successfully
- Use execution logs to see actual data

HTTP Request Fails

Problem: "Request failed with status 404"

Solutions:

- Verify URL is correct
- Check API is accessible
- Confirm authentication headers
- Test API endpoint externally
- Check timeout settings

Email Not Sending

Problem: "Failed to send email"

Solutions:

- Verify SMTP settings in backend/.env
- Check SMTP credentials
- Test SMTP connection
- Verify recipient email
- Check firewall settings

Getting Help

1. **Check Logs:** Review execution logs for errors
2. **Review Documentation:** Check this guide and README
3. **Test Components:** Isolate problematic nodes

4. **Check Console:** Look for browser console errors
5. **Ask for Help:** Create an issue with:
 - Workflow configuration
 - Error messages
 - Steps to reproduce
 - Expected vs actual behavior

Tips & Tricks

Debugging Workflows

1. Add Data Transform nodes to log intermediate values
2. Use Set Variable nodes to store checkpoints
3. Test expressions in isolation
4. Run workflows with simple trigger data first
5. Check execution logs for each node

Reusing Workflows

1. Save successful workflows as templates
2. Export workflow JSON for backup
3. Document complex workflows
4. Share workflow patterns with team

Advanced Techniques

1. **Conditional Branching:** Use Conditional nodes to create different paths
2. **Error Handling:** Add error branches for failures
3. **Data Aggregation:** Combine data from multiple sources
4. **Retry Logic:** Implement custom retry mechanisms

Next Steps

- Create your first workflow
- Explore different node types
- Experiment with expressions
- Monitor execution performance
- Plan more complex workflows

Happy automating! 🚀